

☐ Hide inserted text ☐ Hide deleted text

P1972R0: US105 Check satisfaction of constraints for non-templates when forming pointer to function

This paper presents the wording changes for the NB comment US105 on the C++20 CD.

Do not change [dcl.fct.def.delete] paragraph 2.

Change 7.3.3 [conv.func]:

An lvalue of function type T can be converted to a prvalue of type “pointer to T”. The result is a pointer to the function. [Footnote: ...]

~~[Note: See 12.5 for additional rules for the case where the function is overloaded. — end note]~~

Change in 7.5.4 [expr.prim.id] paragraph 5:

For an *id-expression* that denotes an overload set, overload resolution is performed to select a unique function (12.4 [over.match], 12.5 [over.over]). ~~A program that refers explicitly or implicitly to a function with a trailing *requires-clause* whose constraint-expression is not satisfied, other than to declare it, is ill-formed.~~ **[Note: A program cannot refer to a function with a trailing *requires-clause* whose constraint-expression is not satisfied, because such functions are never selected by overload resolution.** [Example:

```
void f(int) requires false;
void g() {
    f(0); // error: cannot call f
    void (*p1)(int) = f; // error: cannot take the address of f
    decltype(f)* p2 = nullptr; // error: the type decltype(f) is invalid
}
```

In each case, the constraints of f are not satisfied. In the declaration of p2, those constraints are required to be satisfied even though f is an unevaluated operand (7.2). — end example] **-- end note]**

Change in 7.6.1.2 [expr.call] paragraphs 1 and 3:

A function call is a postfix expression followed by parentheses containing a possibly empty, comma-separated list of initializer-clauses which constitute the arguments to the function. **[Note: If the postfix expression is a function or member function name, the appropriate function and the validity of the call are determined according to the rules in 12.4 [over.match]. -- end note]** The postfix expression shall have function type or function pointer type. For a call to a non-member function or to a static member function, the postfix expression shall either be an lvalue that refers to a function (in which case the function-to-pointer standard conversion (7.3.3) is suppressed on the postfix expression), or have function pointer type.

...

~~If a function or member function name is used, the appropriate function and the validity of the call are determined according to the rules in 12.4.~~ If the selected function is non-virtual, or if the id-

expression in the class member access expression is a qualified-id, that function is called. Otherwise, its final overrider (11.7.2) ...

Change in 7.6.2.1 [expr.unary.op] paragraph 6:

[Note: The address of an overloaded function (Clause 12) can be taken only in a context that uniquely determines which version of the overloaded function is referred to (see 12.5). ~~[Note:~~ Since the context might determine whether the operand is a static or non-static member function, the context can also affect whether the expression has type “pointer to function” or “pointer to member function”. — end note]

Change in 12.5 [over.over] paragraph 1:

A use of ~~an overloaded~~ **a** function name without arguments is resolved ~~in certain contexts~~ to a function, a pointer to function or a pointer to member function for a specific function from **a set of selected functions, as described below** ~~the overload set. A function template name is considered to name a set of overloaded functions in such contexts.~~ A function with type F is selected for the function type FT of the target type required in the context if F (after possibly applying the function pointer conversion (7.3.13)) is identical to FT. [Note: That is, the class of which the function is a member is ignored when matching a pointer-to-member-function type. — end note] The target can be

- ...

If there is no target, all functions named are selected. The ~~overloaded~~ function name can be preceded by the & operator. ~~An overloaded function name shall not be used without arguments in contexts other than those listed.~~ [Note: Any redundant set of parentheses surrounding the ~~overloaded~~ function name is ignored (7.5.3). — end note]

Change in 12.5 [over.over] paragraph 2:

For each function template designated by the name, ~~If the name is a function template,~~ template argument deduction is done (13.10.2.2), and if the argument deduction succeeds, the resulting template argument list is used to generate a single function template specialization, which is added to the set **of selected functions** ~~of overloaded functions~~ considered. [Note: As described in 13.10.1, if deduction fails and the function template name is followed by an explicit template argument list, the template-id is then examined to see whether it identifies a single function template specialization. If it does, the template-id is considered to be an lvalue for that function template specialization. The target type is not used in that determination. — end note]

Change 13.10.2.2 [temp.deduct.funcaddr]:

Template arguments can be deduced from the type specified when taking the address of an overloaded function (12.5). **If there is a target, the** ~~The~~ function template’s function type and the ~~specified target~~ type are used as the types of P and A, and the deduction is done as described in 13.10.2.5. **Otherwise, deduction is performed with empty sets of types P and A.**

A placeholder type (9.2.8.5) in the return type of a function template is a non-deduced context. If template argument deduction succeeds for such a function, the return type is determined from instantiation of the function body.